

# PROJECT REPORT

## PLANTS VS ZOMBIES

### Java Swing Game Development

Group: AB

Member: (Student - ID)

Thái Nhật Anh – ITCSIU25004

#### Abstract

This project implements a simplified version of the popular tower defense game **Plants vs. Zombies** using Java and Swing. The game features multiple plant types, different zombie variants, projectile mechanics, resource (sun) management, and defensive systems such as Lawn Mowers. The objective is to defend the lawn from waves of zombies using strategic plant placement. The project demonstrates object-oriented programming principles, GUI development, animation handling, and collision detection.

#### 1. Introduction

Plants vs. Zombies is a classic tower defense game where players must prevent zombies from reaching their house by planting defensive plants. This project recreates the core gameplay mechanics in Java using the Swing library for the graphical user interface.

Objectives:

- Implement core game mechanics (planting, shooting, zombie movement, collision).
- Apply Object-Oriented Programming (OOP) concepts effectively.
- Create an engaging and responsive graphical game.
- Demonstrate proper software design through class hierarchy and inheritance.

#### 2. Game Rules

- The game is played on a lawn with multiple horizontal lanes.
- **Sun** is the main resource. It is generated by **Sunflowers** and appears randomly.
- Players click on the sun to collect it and use it to buy plants from the menu (not fully implemented in current version).

- **Plants** are placed on the grid to defend against incoming **Zombies**.
- Zombies move from right to left. If any zombie reaches the left side ( $x = 0$ ), the player loses.
- Different plants have unique abilities:
- **Peashooter**: Shoots peas at zombies.
- **Sunflower**: Produces suns over time.
- **Wall-nut**: High health defensive wall.
- **Potato Mine**: Explodes when a zombie steps on it.
- **Lawn Mowers** automatically activate and clear an entire lane when zombies get too close.
- The game ends when all lawn mowers are used and a zombie reaches the house.

### 3. System Design

#### 3.1 Class Diagram

The system follows a clean object-oriented design with proper use of inheritance and composition.

Main Class Hierarchy:

##### - Plant (Abstract Class)

+ Peashooter

+ Sunflower

+ Wallnut

+ PotatoMine

##### - Zombie (Base Class)

+ Normal Zombie

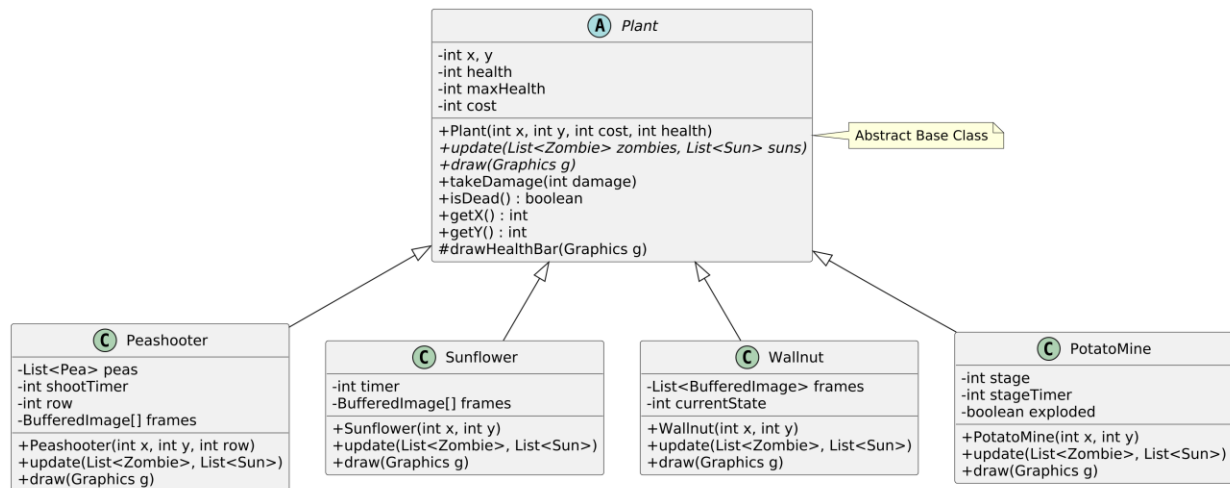
+ ConeheadZombie

+ BucketheadZombie

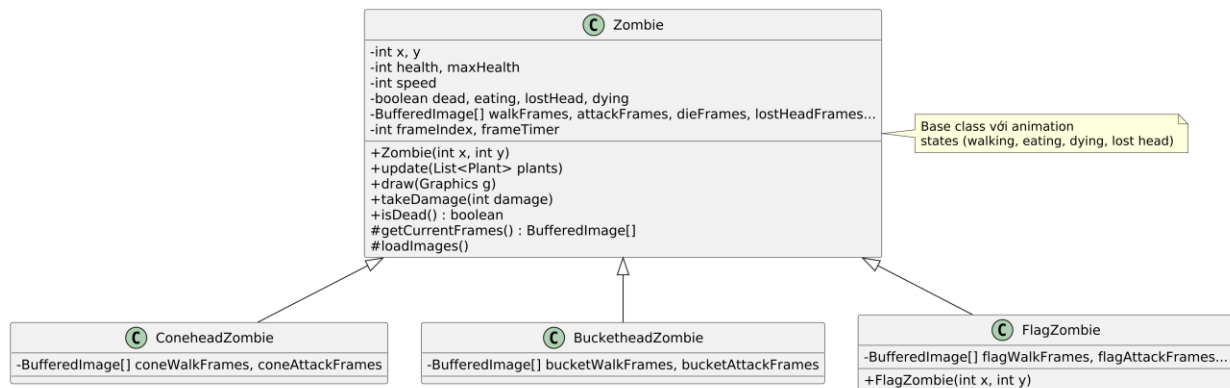
+ FlagZombie

##### - Supporting Classes: **Pea, Sun, LawnMower, GamePanel, MenuPanel**

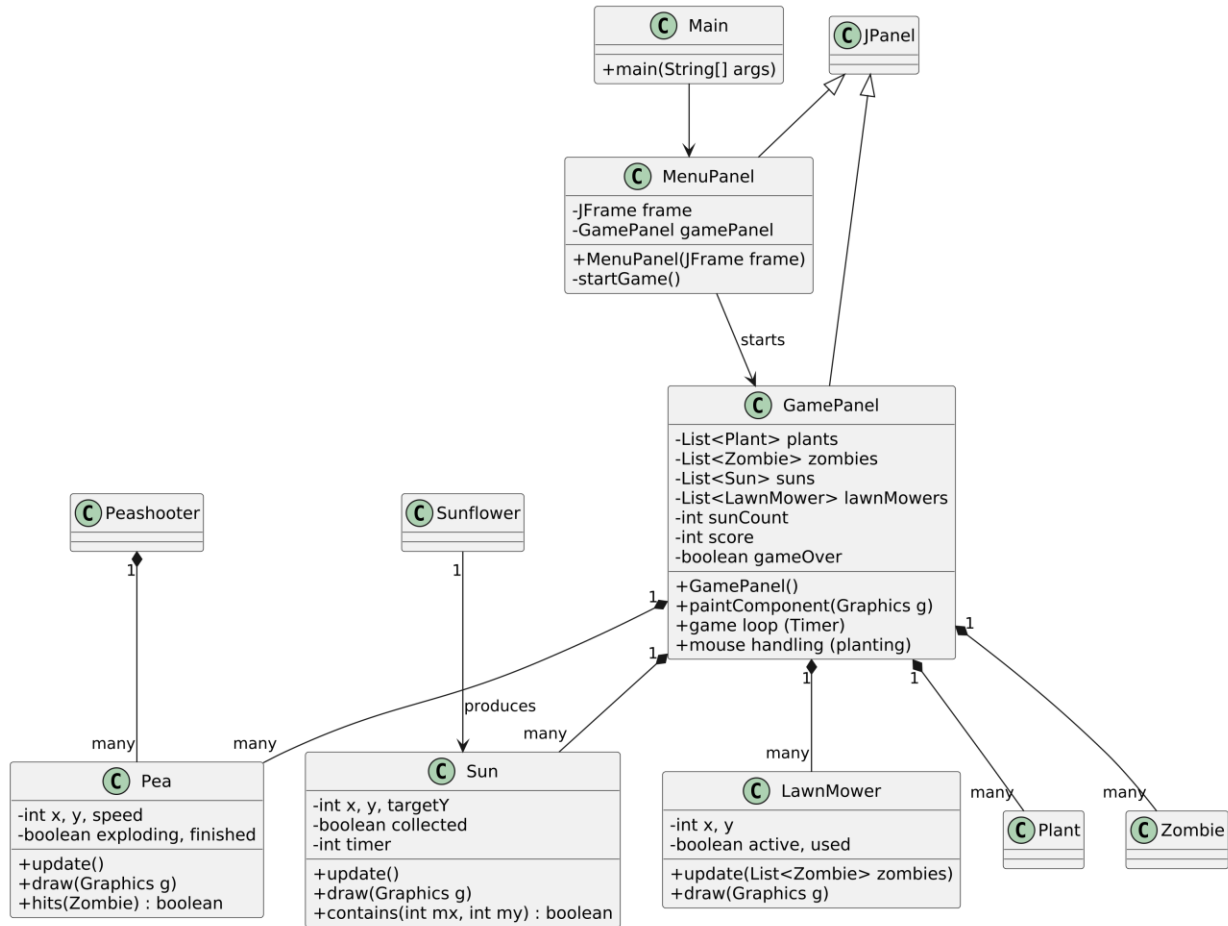
Class diagram Plants:



### Class diagrams Zombies:



### Class Diagram - Game Core & Supporting Classes



## 3.2 Design Patterns Used

- Inheritance & Polymorphism: Heavy use in **Plant** and **Zombie** hierarchies.
- Composition: **GamePanel** contains lists of plants, zombies, suns, etc.
- Factory-like pattern (partial): Different zombie types are created with different properties.

## 4. Implementation

### 4.1 Technologies Used

- Language: Java
- GUI Framework: Swing
- Image Handling: ``BufferedImage`` + ``ImageIcon``
- Animation: Frame-based animation with timers

## 4.2 Key Features Implemented

- **Main Menu:** Play & Exit buttons
- **Multiple Plant Types:** Peashooter, Sunflower, Wallnut, Potato Mine
- **Multiple Zombie Types:** Normal, Conehead, Buckethead, Flag
- **Shooting Mechanics:** Peas with collision
- **Sun Collection System:** Sun spawning & collection
- **Lawn Mower Defense:** Auto-activation
- **Health & Damage System:** Progressive damage (cracked Wall-nut, etc.)
- **Animation System:** Smooth frame animation for all entities
- **Collision Detection:** Plant-Zombie, Pea-Zombie, Mine-Zombie

## 4.3 Important Classes

- **GamePanel.java** (Main game loop – currently missing in provided files, needs implementation)
- **Plant.java** – Abstract base class
- **Zombie.java** – Base zombie with animation states
- **Peashooter.java** – Shooting logic
- **Sunflower.java** – Resource generation
- **LawnMower.java** – Emergency defense

## 5. Challenges & Solutions

### 1. Animation Performance

Challenge: Smooth animation with many entities.

Solution: Used frame counters and **Graphics2D** with interpolation.

### 2. Collision Detection

Challenge: Accurate hit detection.

Solution: Used distance-based and rectangle overlap checks.

### 3. Resource Loading

Challenge: Loading many image files.

Solution: Created robust **loadResourceImage()** method with multiple path fallbacks.

### 4. Game Loop

Challenge: Managing updates and rendering.

Solution: Used **Timer** in **GamePanel** (recommended).

### 6. Future Improvements (Bonus Features)

- Add plant selection bar on top
- Grid-based planting system
- Multiple levels / increasing difficulty
- Sound effects
- Save/Load game progress
- More plant and zombie types
- Win condition (survive X waves)

### 7. Conclusion

This project successfully demonstrates the implementation of a complex 2D game using Java Swing. Through effective use of OOP principles such as inheritance, polymorphism, and encapsulation, we created a functional and visually appealing version of Plants vs. Zombies. The codebase is modular and extensible, making it easy to add new features.

The development process improved our understanding of game programming, GUI development, and software design.